Department of Electrical and Computer Engineering

Project Report

Project Title: Brain Tumor MRI Image Classification Using CNN

Course Code: CSE101

Section: 05

Submitted To: Reeshoon Sayera (RSY)

Submitted By:

Name	ID
MD. Arif Ali Robin	2312760042

Description

This project solves the problem of classifying brain MRI images into tumor categories using a convolutional neural network. The model learns visual patterns from MRI scans and predicts whether the image belongs to glioma, meningioma, pituitary tumor, or no tumor.

Tools and Libraries

Tool or Library	Used For	Code Explanation
Python	Main programming language	Controls the complete machine learning workflow.
PyTorch torch	Tensor operations and model execution	Handles tensors, moves data to CPU or GPU, and runs the neural network.
torch.nn	CNN model building	Creates convolution layers, activation layers, pooling layers, dropout, flattening, and linear layers.
torch.optim Adam	Model optimization	Updates model weights using the Adam optimizer with learning rate 0.0001.
torchvision.datasets ImageFolder	Dataset loading	Reads Training and Testing folders where subfolders represent class labels.
torchvision.transforms	Image preprocessing	Resizes MRI images to 128 by 128, converts them to tensors, and normalizes pixel values.
DataLoader	Batch processing	Loads images in batches of 32 and shuffles the training data.
Matplotlib	Visualization	Displays sample MRI images and is used here to show the training loss curve.

Key Features

- Loads brain MRI images from Training and Testing folders using PyTorch ImageFolder.
- Preprocesses every image by resizing to 128 x 128, converting to tensor format, and normalizing RGB channels.
- Builds a three-block convolutional neural network with Conv2D, ReLU, MaxPool2D, Flatten, Linear, and Dropout layers.
- Trains the CNN for 25 epochs using CrossEntropyLoss and Adam optimizer.
- Evaluates the model on the test set and calculates test loss and test accuracy.
- Includes a sample prediction cell, which predicts the type of brain tumor.

Code Part Explanation:

1. Data Loading and Preprocessing

The first code cell imports PyTorch, selects CPU or GPU, defines image transformations, and creates train and test DataLoaders. Images are resized to 128 x 128 and normalized before training.

```
import torch
import torch.nn as nn
from torch import optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')


tf = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])


train_dl = DataLoader(
    datasets.ImageFolder('data/Training',tf),
    batch_size=32,
    shuffle=True,
    num_workers=4,
    pin_memory=True
)

test_dl = DataLoader(
    datasets.ImageFolder('data/Testing',tf),
    batch_size=32,
    shuffle=False,
    num_workers=4,
    pin_memory=True
)
```

✓ 9.4s

2. CNN Model Architecture

The model has three convolution blocks. Each block extracts image features using convolution, applies ReLU activation, and reduces image size using max pooling. The extracted features are flattened and passed through fully connected layers for four-class classification.

```
model = nn.Sequential(
    nn.Conv2d(3,32,3,1,1),nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32,64,3,1,1),nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(64,128,3,1,1),nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(128*16*16, 256), nn.ReLU(),nn.Dropout(0.5),
    nn.Linear(256, 4)
).to(device)
```
✓ 0.0s

3. Training Loop

The training loop runs for 25 epochs. For every batch, the optimizer resets gradients, the model predicts outputs, loss is calculated, backpropagation is performed, and the optimizer updates model weights.

```
model.train()
for epoch in range(25):
    running_loss = 0
    for x,y in train_dl:
        opt.zero_grad()

        loss = loss_fn(model(x.to(device)),y.to(device))
        loss.backward()

        running_loss += loss
        opt.step()
    print(f'Epoch {epoch+1}:Loss was {running_loss}')
```

4. Evaluation Code

The evaluation code turns off gradient calculation, predicts labels for test images, calculates average test loss, counts correct predictions, and prints final accuracy.

```
model.eval()
test_loss,correct = 0.0,0
with torch.no_grad():
    for x,y in test_dl:
        x,y = x.to(device),y.to(device)

        logits = model(x)
        test_loss += loss_fn(logits,y).item()*y.size(0)

        preds = logits.argmax(dim=1)
        correct += (preds == y).sum().item()
test_loss /= len(test_dl.dataset)
accuracy = 100.0*correct/len(test_dl.dataset)
print('test Loss:',test_loss, 'Test Accuracy:',accuracy,'%')
```

Output:

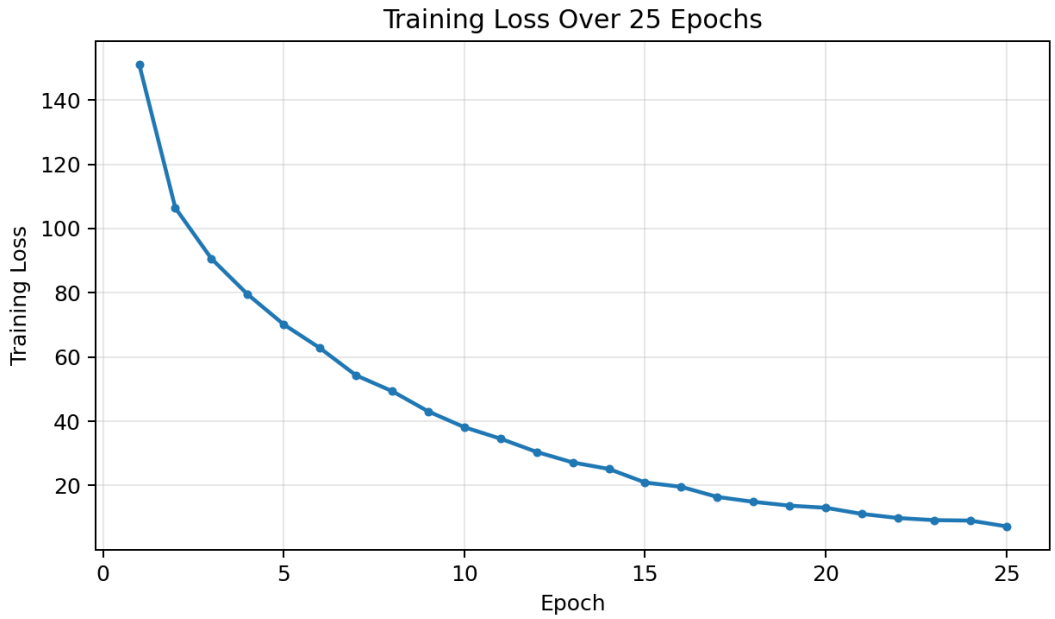
Final evaluation result: Test Loss = 1.33098, Test Accuracy = 89.9375 percent.

Metric	Value
Training epochs	25
Final training loss	8.316158294677734
Test loss	1.3309822135214926
Test accuracy	89.9375 percent
Sample prediction class	Glioma
Sample true class	Glioma

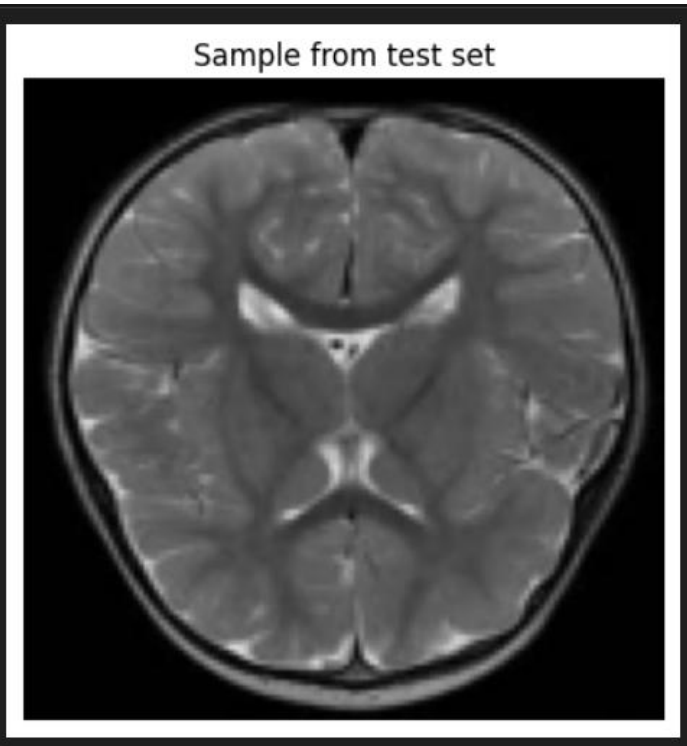
```
c:\Users\Robin\Desktop\101\.venv\Lib\site-packages\torch\utils\data\dataloader.py:1118: UserWar
  super().__init__(loader)
Epoch 1:Loss was 152.9169921875
Epoch 2:Loss was 101.02676391601562
Epoch 3:Loss was 86.84346771240234
Epoch 4:Loss was 76.21537780761719
Epoch 5:Loss was 66.71587371826172
Epoch 6:Loss was 58.004512786865234
Epoch 7:Loss was 51.99906539916992
Epoch 8:Loss was 46.850887298583984
Epoch 9:Loss was 42.094703674316406
Epoch 10:Loss was 34.70824432373047
Epoch 11:Loss was 32.674163818359375
Epoch 12:Loss was 27.253273010253906
Epoch 13:Loss was 24.72837257385254
Epoch 14:Loss was 22.20865821838379
Epoch 15:Loss was 19.833600997924805
Epoch 16:Loss was 19.093475341796875
Epoch 17:Loss was 15.131905555725098
Epoch 18:Loss was 13.877670288085938
Epoch 19:Loss was 12.663362503051758
Epoch 20:Loss was 12.35538101196289
Epoch 21:Loss was 10.083806037902832
Epoch 22:Loss was 8.799322128295898
Epoch 23:Loss was 9.27386474609375
Epoch 24:Loss was 8.594337463378906
Epoch 25:Loss was 8.316158294677734
```

```python
model.eval()
test_loss,correct = 0.0,0
with torch.no_grad():
    for x,y in test_dl:
        x,y = x.to(device),y.to(device)

        logits = model(x)
        test_loss += loss_fn(logits,y).item()*y.size(0)

        preds = logits.argmax(dim=1)
        correct += (preds == y).sum().item()
test_loss /= len(test_dl.dataset)
accuracy = 100.0*correct/len(test_dl.dataset)
print('test Loss:',test_loss, 'Test Accuracy:',accuracy,'%')
```

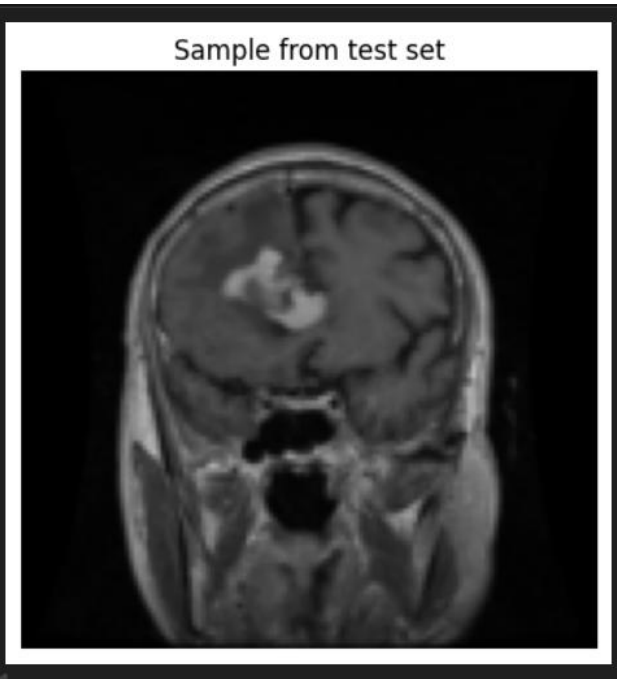
```
test Loss: 1.3309822135214926 Test Accuracy: 89.9375 %
```



Training loss decreased from 152.9169 in epoch 1 to 8.3161 in epoch 25, which shows that the model learned useful image patterns during training.




```
Predicted Class: notumor
True Class: notumor
```




```
Predicted Class: glioma
True Class: glioma
```

Challenges and Limitations

During this project, one of the main challenges was working with MRI image data because the images needed to be resized, normalized, and properly organized before training the model. Since the dataset contains different brain tumor categories, the model had to learn visual patterns from medical images, which can be difficult when the differences between classes are small. Another limitation is that the model was trained only on the available Kaggle dataset, so its performance may not be the same on real hospital MRI scans or completely new datasets. The model also depends on the quality and balance of the training images. If some classes have fewer or less clear images, the prediction accuracy may be affected. The CNN model achieved good accuracy, but it is still a basic deep learning model. It may make incorrect predictions in complex cases. Also, the model does not explain why it predicts a specific tumor class, so it should not be used as a final medical decision system. It can only be used as a support tool for learning and research purposes.

Conclusion

This project successfully developed a convolutional neural network model to classify brain MRI images into different tumor categories. The model was trained using Python and PyTorch, and the final evaluation showed that it could classify test images with good accuracy. The project demonstrates how deep learning can be applied to medical image classification tasks.The system can be improved in the future by using a larger and more diverse dataset, applying data augmentation, tuning hyperparameters, and using advanced pre-trained models such as ResNet, VGG, or EfficientNet. More evaluation methods, such as a confusion matrix and precision, recall, and F1-score, can also be added to better understand the model performance.